

Creating the Maze Data Structure

The maze data structure is a Python dictionary or JavaScript object that has keys of Python tuples or JavaScript arrays of the x, y coordinates of every space in the maze. The value for these keys is a string in the `WALL` or `EMPTY` constant. This string notes whether this space is a blocking wall or a passable empty space in the maze.

For example, the maze in [Figure 11-2](#) is represented by the following data structure:

```
{(0, 0): '■', (0, 1): '■', (0, 2): '■', (0, 3): '■', (0, 4): '■',
(0, 5): '■', (0, 6): '■', (1, 0): '■', (1, 1): ' ', (1, 2): ' ',
(1, 3): ' ', (1, 4): ' ', (1, 5): ' ', (1, 6): '■', (2, 0): '■',
(2, 1): '■', (2, 2): '■', (2, 3): '■', (2, 4): '■', (2, 5): ' ',
(2, 6): '■', (3, 0): '■', (3, 1): ' ', (3, 2): '■', (3, 3): ' ',
(3, 4): ' ', (3, 5): ' ', (3, 6): '■', (4, 0): '■', (4, 1): ' ',
(4, 2): '■', (4, 3): ' ', (4, 4): '■', (4, 5): '■', (4, 6): '■',
(5, 0): '■', (5, 1): ' ', (5, 2): ' ', (5, 3): ' ', (5, 4): ' ',
(5, 5): ' ', (5, 6): '■', (6, 0): '■', (6, 1): '■', (6, 2): '■',
(6, 3): '■', (6, 4): '■', (6, 5): '■', (6, 6): '■'}
```

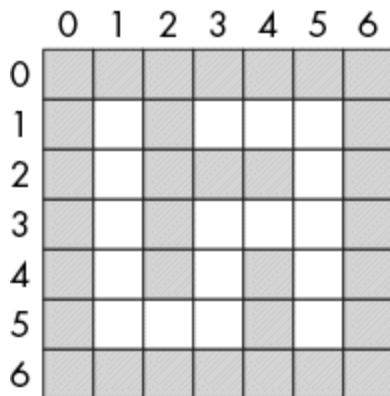


Figure 11-2: An example maze that can be represented by a data structure

The program must start with every space set to `WALL`. The recursive `visit()` function then carves out the hallways and intersections of the maze by setting spaces to `EMPTY`:

Python